

计算机组成原理实验报告

实验名称：算术逻辑单元 ALU 设计

专业班级：计算机科学与技术 一班

组 员：王之翼 张潘妍 张之恒 陈海攀

指导教师：-

实验日期：2026 年 3 月 28 日

兰州大学信息科学与工程学院

1. 设计功能

本实验目标是完成一个 32 位算术逻辑单元（ALU）的模块化设计，采用 one-hot 控制编码，部分功能采用时序逻辑运算。

本实验项目不依赖 IP 核，使用接近于 verilog 原语的方法进行构建。

具体实现的功能如下：

- 32 位基本算术运算：ADD、SUB。
- 32 位逻辑运算：AND、OR、NOT、XOR、NOR。
- 比较运算：SLT（有符号小于）、SLTU（无符号小于）。
- 移位运算：SLL、SRL、SRA（左/右逻辑移位，右算数移位）。
- 立即数高位加载：LUI（将 src2[15:0] 装载到结果高 16 位）。
- 多周期乘除法：MUL（Booth 有符号乘法）与 DIV（非恢复余数法有符号除法）。
- 完成标志 done：组合逻辑类操作单周期完成，乘除法在状态机结束时置位。

2. 功能原理

2.1. 顶层控制与数据通路

顶层模块输入为 clk、reset、alu_control[15:0]、src1[31:0]、src2[31:0]，输出为 result[31:0] 与 done。控制信号采用 one-hot 编码：

码（左低右高）	功能
0000 0000 0000 000 x	—
0000 0000 0000 00 10	LUI
0000 0000 0000 0 100	SRA
0000 0000 0000 1000	SRL
0000 0000 000 1 0000	SLL
0000 0000 00 10 0000	XOR
0000 0000 0 100 0000	OR
0000 0000 1000 0000	NOR

码（左低右高）	功能
0000 000 1 0000 0000	AND
0000 00 10 0000 0000	SLTU
0000 0 100 0000 0000	STL
0000 1000 0000 0000	SUB
000 1 0000 0000 0000	ADD
00 10 0000 0000 0000	NOT
0 100 0000 0000 0000	DIV
1000 0000 0000 0000	MUL

输入数据经过所有模块的并行计算后通过 alu_result_selector 多选一模块选择输出。

对于多周期操作的同步问题，我们通过 done 信号线进行完成控制，且区分了组合操作和时序操作的 done 信号。

2.2. 超前进位加法器 (CLA)

对于一般的行波进位加法器中的每一个组件 (全加器), 我们有:

$$\begin{cases} S_i = A \oplus B \oplus C_i \\ C_{i+1} = A_i \cdot B_i + A_i \cdot C_i + B_i \cdot C_i \end{cases}$$

令 $G_i = A_i \cdot B_i$, $P_i = A_i + B_i$, 则 $C_{i+1} = G_i + P_i \cdot C_i$ 。

可见, 进位中的 G_i 和 P_i 不依赖前面的数据, 仅依赖本位数据, 于是, 我们可以将行波进位加法器改为:

$$\begin{cases} C_1 = G_0 + P_0 \cdot C_{in} \\ C_2 = G_1 + P_1 \cdot C_1 \\ \quad = G_1 + P_1 \cdot G_0 + P_0 \cdot P_1 \cdot C_{in} \\ C_3 = G_2 + P_2 \cdot C_2 \\ \quad = G_2 + P_2 \cdot G_1 + P_1 \cdot P_2 \cdot G_0 + P_0 \cdot P_1 \cdot P_2 \cdot C_{in} \\ C_4 = G_3 + P_3 \cdot C_3 \\ \quad = G_3 + P_3 \cdot G_2 + P_2 \cdot P_3 \cdot G_1 + P_1 \cdot P_2 \cdot P_3 \cdot G_0 + P_0 \cdot P_1 \cdot P_2 \cdot P_3 \cdot C_{in} \\ C_{out} = C_4 \end{cases}$$

可见, 这个四位行波进位加法器其实不需要一个一个计算, 每一个进位实际上都可以直接通过输入数据进行计算。

又由于 $S_i = A \oplus B \oplus C_i$, 我们可以构建一个四位超前进位加法器。

32 位超前进位加法器虽然也可以直接构建 (通过找规律), 但是这样对于芯片面积的消耗太大, 也不利于布线, 所以我们加法模块采用 32 位层次化超前进位结构: 4 位 CLA 组成 16 位 CLA, 再由两个 16 位 CLA 构成 32 位加法器。其核心思想是提前并行计算进位, 而不是像行波进位那样逐位传播。

对第 i 位定义:

- 进位生成信号: $G_i = A_i \cdot B_i$
- 进位传递信号: $P_i = A_i \oplus B_i$
- 进位关系: $C_{i+1} = G_i + P_i C_i$
- 和位: $S_i = P_i \oplus C_i$

CLA 的优点是进位计算层次化并行, 可显著降低长位宽加法延迟。顶层 ADD 直接取 CLA 输出。

2.3. 桶形移位器

固定位的位移很简单, 就是将输入输出线对应连接好就行, 见 SHL_x.v, 问题在于不固定位数的位移器。

参考人民币币值的设计, 不固定位数的位移器也可以使用多个固定位的位移完成。

在设计上来说，以 2^n 作为划分是一个很好的想法，一方面每个位移最多只需要一个就可以完成任意大小的位移，另一方面可以直接读取二进制位判断需不需要当前位数的位移。

具体来说，移位模块采用 5 级桶形位移结构，每一级对应一个二次幂移位量：1、2、4、8、16。通过 `shamt[4:0]` 控制各级是否生效，最终实现 0 到 31 位任意移位。

2.4. 小于置位 (SLT/SLTU) 判断方法

SLT 与 SLTU 都输出 32 位，其中低位 `bit0` 为比较结果，其余位清零。

1. SLT (有符号小于)

先做减法 $a - b$ ，然后分情况判断：

- 若 a 与 b 符号不同， a 为负时必有 $a < b$ ；
- 若符号相同，则比较差值符号位。

代码等价表达为：

$$\text{slt} = (a_{31} \wedge \overline{b_{31}}) \vee (\overline{a_{31} \oplus b_{31}} \wedge \text{sub}_{31})$$

2. SLTU (无符号小于)

无符号比较通过减法借位判断，而由于是补码表示，减法转化为加法，所以最终是比较进位。

令 $a + (\sim b + 1)$ 的最终进位为 `cout`：

- `cout = 1` 表示无借位，即 $a \geq b$ ；
- `cout = 0` 表示有借位，即 $a < b$ 。

因此 `sltu = ~cout`。

2.5. 乘法算法 (Booth)

Booth 乘法器实际上是使用 Booth 编码的乘法器，布斯编码可以减少部分积的数目，用来计算有符号乘法，提高乘法运算的速度。

考虑一个由若干个 0 包围着若干个 1 的正的二进制乘数，比如 00111110，与另一个乘数 M 的积可以表达为：

$$M \times [00111110]_B = M \times (2^5 + 2^4 + 2^3 + 2^2 + 2^1) = M \times 62$$

按照类手算算法需要至少 5 次乘法计算，而乘法而变形为下面的方式可以让计算次数减少为两次：

$$M \times [010000(-1)0]_B = M \times (2^6 - 2^1) = M \times 62$$

事实上,任何二进制数中连续的 1 可以被分解为两个二进制数之差:

$$\left(\overbrace{\dots 01\dots 10\dots}^n \right)_B = \left(\overbrace{\dots 10\dots 00\dots}^n \right)_B - \left(\overbrace{\dots 00\dots 10\dots}^n \right)_B$$

因此,我们可以用更简单的运算来替换原数中连续为 1 的数字的乘法,通过加上乘数,对部分积进行移位运算,最后再将之从乘数中减去。它利用了我们在针对为零的位做乘法时,不需要做其他运算,只需移位这一特点,这很像我们在做和 99 的乘法时利用 $99=100-1$ 这一性质。这种模式可以扩展应用于任何一串数字中连续为 1 的部分(包括只有一个 1 的情况)。

布斯算法遵从这种模式,它在遇到一串数字中的第一组从 0 到 1 的变化时(即遇到 01 时)执行加法,在遇到这一串连续 1 的尾部时(即遇到 10 时)执行减法。这在乘数为负时同样有效。当乘数中的连续 1 比较多时(形成比较长的 1 串时),布斯算法较一般的乘法算法执行的加减法运算少。

2.6. 非恢复余数除法器

非恢复余数除法器是恢复余数除法器的优化版本,通过延迟恢复与后续操作合并的操作,节省了每一步的恢复步骤,并且数据通路更加连贯,复用性更高,使得其拥有更高的性能。

算法步骤(以 n 位运算为例)

1. 初始化

- 将除数存入寄存器 Y (n 位补码)。
- 将 $-Y$ 存入寄存器(取补码)。
- 将被除数符号扩展后装入 R (高 n 位) 和 Q (低 n 位)。

2. 循环迭代(共 $n-1$ 次)

对于 $i = 1$ 到 $n-1$:

- 左移: 将 (R, Q) 组合左移一位, 相当于 $R = 2R$ 并移入 Q 的最高位, Q 左移一位。
- 运算选择:
 - 若当前 R_i 与 Y **同号**, 则 $R_{i+1} = 2R_i - Y$ (减法);
 - 若当前 R_i 与 Y **异号**, 则 $R_{i+1} = 2R_i + Y$ (加法)。
- 上商:
 - 若新余数 R_{i+1} 与 Y **同号**, 则商位为 1;
 - 否则商位为 0。
- 将商位移入 Q 的最低位。

3. 第 n 次操作

- 完成第 n 次左移和加减后，仅将商位写入 Q ，不再对 R 进行下一轮更新（因为已得到 n 位商）。

4. 商修正

- 若**被除数**与**除数**异号，则最终的商需要加 1（即对未修正的商求补码）。
- 若同号，商不变。

5. 余数修正

- 若最终**余数** R 与**被除数**同号，则无需修正。
- 若异号：
 - 若**被除数**与**除数**同号，则余数**加**除数；
 - 若被除数与除数**异号**，则余数**减**除数。

6. 特殊情形修正

- **同号相除且能整除**（如 $\frac{-8}{-8}$ ）：若余数等于除数，则余数减去除数，商加 1。
- **异号相除且能整除**（如 $\frac{-8}{2}$ ）：若余数加上除数为 0，则商减 1，余数置 0。

3. 功能实现

3.1. 模块划分

alu_32bit.v 在结构上采用“并行计算 + 统一选择”的实现方式，主要由以下子模块组成（每一个都是独立的.v 文件）：

- cla_adder_32bit: 32 位超前进位加法器顶层模块。
- subtractor: SUB 运算（内部调用 ADD）。
- logic_unit: AND/OR/NOT/XOR/NOR/SLT/SLTU 集成模块。
- shifter 实现 SLL、SRL、SRA。
- lui: 高位装载运算。
- booth_multiplier: 有符号乘法，多周期。
- non_restoring_divider: 有符号除法，多周期。
- alu_result_selector: 对 15 路候选结果进行选择输出。

3.2. 顶层集成流程

实现流程如下：

1. 由 alu_control 解码得到各运算使能位。
2. 组合运算模块并行产生结果。
3. 对 MUL/DIV 在时钟上升沿锁存 src1/src2 到寄存器，顶层只发出单周期 start 脉冲。

4. 顶层使用 busy + req_armed 机制抑制重触发：同一操作码保持为高时不会重复启动，必须先释放操作码再允许下一次启动。
5. 乘法在 mul_done 时锁存低 32 位结果，除法在 div_done 时锁存商值；result 选择器在 MUL/DIV 模式下输出锁存值而不是计算中间值。
6. done 在 MUL/DIV 模式下采用 done_pulse OR done_hold 稳定语义，避免在板级显示中出现高频闪烁。
7. 非乘除法仍保持组合路径 done = 1，从而与多周期路径形成统一的接口语义。

本次调试中遇到的核心问题是：乘除法初版启动条件是电平触发，alu_control 长时间保持 MUL/DIV 时会在一次完成后自动再次启动，导致 result 与 done 在显示端看起来持续刷新。修复经验总结如下：

- 多周期单元的 start 应设计为“单拍脉冲 + 忙闲握手”，不要直接由电平控制信号驱动。
- 顶层输出给外设（显示、总线）时，应优先输出完成态锁存值，避免直接暴露迭代中间态。
- 人机交互层应提供显式“释放/重触发”路径。本实验在 FPGA 顶层增加了 SW2(op_clear)：SW2=1 清零操作码，SW2 从 1 -> 0 自动恢复上次非零操作码，便于连续更换数据后重复同类乘除法测试。

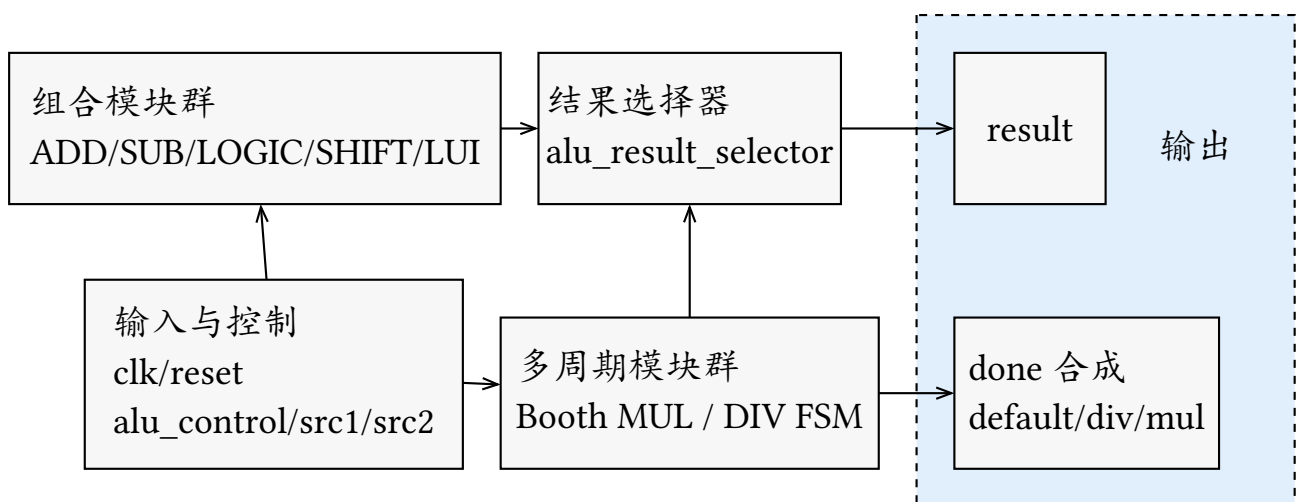


图 1: ALU 顶层模块架构图

3.3. 桶形移位器

移位模块采用 5 级桶形位移结构，每一级对应一个二次幂移位量：1、2、4、8、16。通过 shamt[4:0] 控制各级是否生效，最终实现 0 到 31 位任意移位。

具体实现方式如下：

- 第 1 级根据 shamt[0] 决定是否移 1 位。
- 第 2 级根据 shamt[1] 决定是否移 2 位。

- 第 3 级根据 shamt[2] 决定是否移 4 位。
- 第 4 级根据 shamt[3] 决定是否移 8 位。
- 第 5 级根据 shamt[4] 决定是否移 16 位。

需要注意的是，我们硬件上将 SLL/SRL/SRA 作为三条独立的部件进行实现，功能分别是：

- SLL：左移，低位补 0。
- SRL：右移，高位补 0。
- SRA：算术右移，高位补符号位。

最终使用一个 mux_4to1 部件由 shift_type 在三类结果中选择输出。

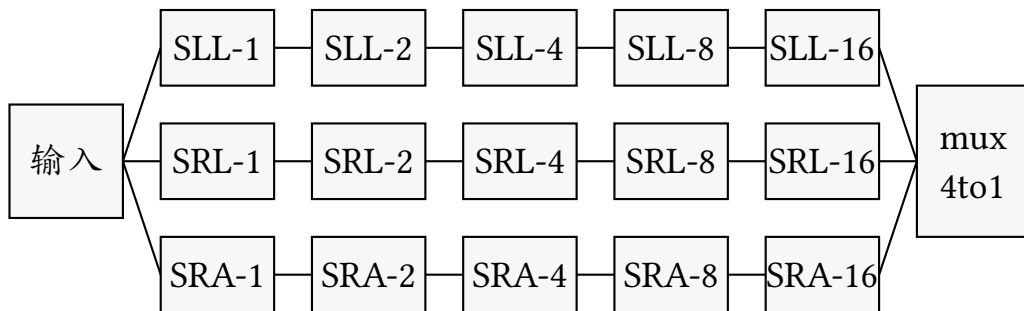


图 2: 桶形移位器

3.4. 超前进位加法器 (CLA)

4 位超前进位加法器完全和原理部分列出的表达式相同，同时它们也输出内部的进位计算结果 G,P，所以 16 位的 CLA 就可以基于其构建：

```
module cla_adder_16bit(
    input  [15:0] a,
    input  [15:0] b,
    input          cin,
    output [15:0] sum,
    output          cout
);
    wire [3:0] g0, p0, g1, p1, g2, p2, g3, p3;
    wire c4, c8, c12;
    cla_adder_4bit cla0(.a(a[3:0]), .b(b[3:0]), .cin(cin),
        .sum(sum[3:0]), .cout(c4), .g(g0), .p(p0));
    cla_adder_4bit cla1(.a(a[7:4]), .b(b[7:4]), .cin(c4),
        .sum(sum[7:4]), .cout(c8), .g(g1), .p(p1));
    cla_adder_4bit cla2(.a(a[11:8]), .b(b[11:8]), .cin(c8),
        .sum(sum[11:8]), .cout(c12), .g(g2), .p(p2));
    cla_adder_4bit cla3(.a(a[15:12]), .b(b[15:12]), .cin(c12),
        .sum(sum[15:12]), .cout(cout), .g(g3), .p(p3));
endmodule
```

代码 1: 16 位超前进位加法器构建

需要注意的是，32 位我们没有使用进位生成器，而是使用了一种类似于串行的方法，因为这一层只需要 2 个 16 位超前进位加法器，时延问题并不明显。

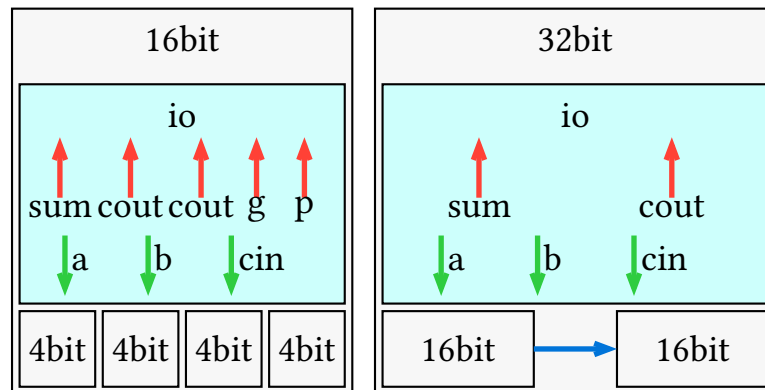


图 3: 超前进位加法器层间组合结构，图中展示了高层级和低层级之间的 IO 联系，32bit 实际上是串联了两个 16bit 组成

3.5. 多路选择器 (MUX)

多路选择器的基础-二路选择器是由门构造的，更高层次的 2^n 选 1 选择器是由 2-1 选择器构造的。

并且，我们使用了 Verilog-2001 标准引入的参数化模块定义语法（类似于模板函数），使得其拥有更强的适应性。

```
module mux_2to1 #(parameter WIDTH = 32)(
    input  [WIDTH-1:0] a,
    input  [WIDTH-1:0] b,
    input                sel,
    output [WIDTH-1:0] y
);
    wire [WIDTH-1:0] a_masked;
    wire [WIDTH-1:0] b_masked;
    wire [WIDTH-1:0] not_sel_vec;
    // 对每一位独立实现: y[i] = (a[i] & ~sel) | (b[i] & sel)
    genvar i;
    generate
        for (i = 0; i < WIDTH; i = i + 1)
            begin : mux_bit
                not u_not_sel(not_sel_vec[i], sel);
                and u_and_a(a_masked[i], a[i], not_sel_vec[i]);
                and u_and_b(b_masked[i], b[i], sel);
                or  u_or_y(y[i], a_masked[i], b_masked[i]);
            end
    endgenerate
endmodule
```

代码 2: 2-1 选择器

3.6. Booth 乘法器

乘法器采用 Booth 有符号乘法算法，状态机为 IDLE -> COMPUTE -> FINISH。寄存器含义如下：

- A：部分积累加器。
- Q：乘数寄存器。
- Q_1 ：扩展位。
- M：被乘数寄存器。

每个计算周期检查二位组合 ($Q[0], Q_1$)：

- 00 或 11：不加减，仅算术右移。
- 01：执行 $A = A + M$ ，再算术右移。
- 10：执行 $A = A - M$ ，再算术右移。

共迭代 32 次后进入 FINISH，输出 $product = (A, Q)$ ，顶层取低 32 位作为 ALU 的 MUL 结果。

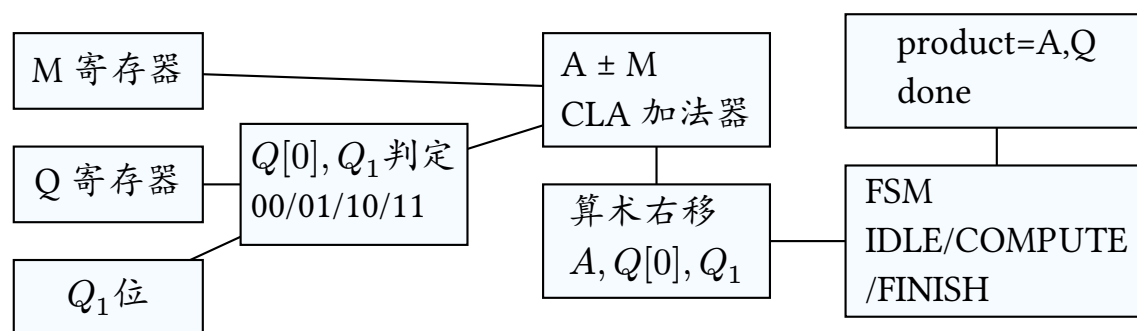


图 4: Booth 乘法器数据通路 with 状态机关系图

3.7. 非恢复余数除法器

除法器采用非恢复余数法 (Non-Restoring Division)，状态机分为 IDLE -> COMPUTE -> FIX -> FINISH 四个阶段。与恢复余数法相比，它主循环中根据当前余数的符号决定执行加法或减法，避免每一步都显式恢复，因此更适合用统一的加减器数据通路实现。

1. 预处理

- 在 IDLE 状态接收 start 后，缓存原始被除数与除数；
- 记录二者符号位，随后将操作数转换为绝对值进入主循环；
- 若除数为 0，则直接进入 FINISH，约定商为 0，余数为被除数；
- 若出现 $INT_MIN / -1$ ，则直接进入 FINISH，按二补码截断语义输出商为 $0x8000_0000$ 、余数为 0。

2. 主循环 (32 次)

- 组合寄存器采用 $\{R, Q\}$ 形式，其中 R 为余数寄存器，Q 为商寄存器；

- 每一轮先整体左移一位，形成 $\text{shifted_R} = \{R[30:0], Q[31]\}$;
- 若当前 R 为非负，则执行 $\text{shifted_R} - D$; 若当前 R 为负，则执行 $\text{shifted_R} + D$;
- 根据新余数的符号位决定本轮商位：新余数非负则 $Q[0] = 1$ ，否则 $Q[0] = 0$ 。

这一过程由 `cla_adder_32bit` 同时构造减法器 and 加法器完成，再通过 MUX 选择下一拍的余数结果。

3. 终态修正

- 32 次迭代结束后进入 FIX 阶段;
- 若最终余数为负，则执行 $R = R + D$ ，使余数回到非负范围;
- 随后进入 FINISH，完成基础非恢复余数运算。

4. 符号恢复与特殊修正

- 商的符号由被除数与除数符号异或得到;
- 余数的符号与被除数相同;
- 若同号除法出现 $\text{remainder} == \text{divisor}$ ，则说明结果可再进一位，模块执行 $\text{quotient} += 1$ 、 $\text{remainder} -= \text{divisor}$;
- 若异号除法出现 $\text{remainder} + \text{divisor} == 0$ ，则执行 $\text{quotient} -= 1$ 、 $\text{remainder} = 0$;
- 对除零和溢出情形，后级直接覆盖最终商和余数。

因此，该除法器并不是简单的“试减-恢复”结构，而是“非恢复余数主循环 + 终态修正 + 特殊商余数校正”的实现方式。最终 `quotient` 作为 ALU 的 DIV 输出，`remainder` 作为附加输出，`done` 在状态机进入 FINISH 后置位。

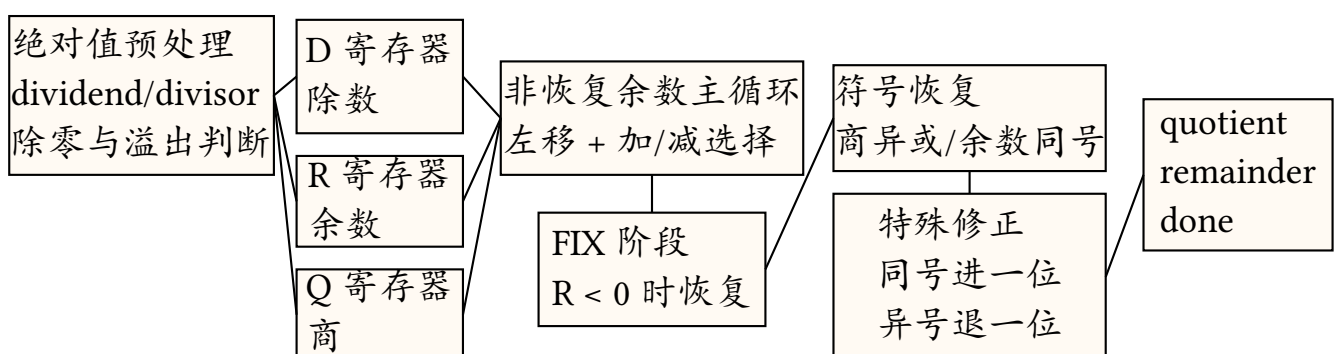


图 5: 非恢复余数除法器数据通路与修正流程图

3.8. 完成信号原理

`done` 信号分两类（高电平有效）：

- 非乘除法操作：`done = 1`（默认立即完成）。
- DIV：`done = div_done or div_done_hold`。

- MUL: done = mul_done or mul_done_hold。

顶层通过多路选择结构根据 alu_control 码选择对应上述信号进行输出，实现统一时序语义。

```

wire mul_done_stable;
wire div_done_stable;
wire done_div_sel;
wire done_default;
assign mul_done_stable = mul_done | mul_done_hold;
assign div_done_stable = div_done | div_done_hold;
assign done_default = 1'b1; // 组合逻辑运算立即完成
mux_2to1 #(1) mux_done_0(.a(done_default), .b(div_done_stable),
    .sel(alu_div), .y(done_div_sel) );
mux_2to1 #(1) mux_done_1(.a(done_div_sel), .b(mul_done_stable),
    .sel(alu_mul), .y(done) );

```

代码 3: done 信号处理

4. 验证

在构建过程中，我们使用 iverilog 进行仿真验证：

命令：python .\mk.py --top .\dev\1alu\tb\tb_alu_32bit.v，结果如下：

```

Test 34: AND - All ones
PASS: AND: 0xFFFFFFFF & 0xFFFFFFFF
Expected: ffffffff, Got: ffffffff

=====
Test Summary
=====
Total tests: 34
Passed:      34
Failed:      0
Pass rate:   100.0%

=====
ALL TESTS PASSED!
E:\Xprogram\FPGA\cpu-designers\dev\1alu\tb\tb_alu_32bit.v:389: $finish called at 3725000 (1ps)
[INFO] Simulation success. Log: E:\Xprogram\FPGA\cpu-designers\build\tb_alu_32bit.run.log

```

Figure 1: 命令行截取，34 测试全部通过

同时也使用 vivado 进行了波形验证：



Figure 2: vivado 波形验证

在实验课上，我们使用 FPGA 实验箱进行了验证，单周期指令均运行良好，例如加法：

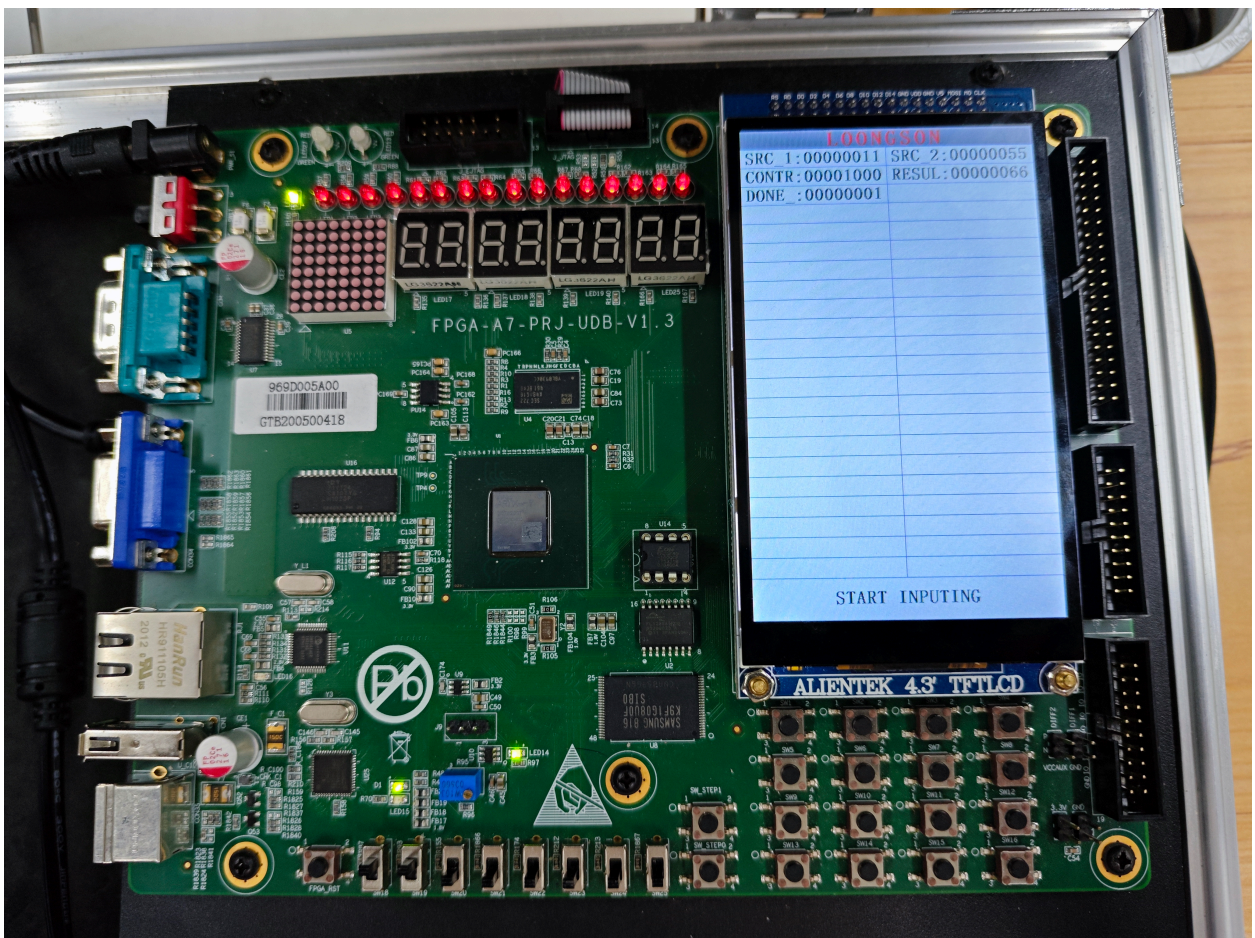


Figure 3: 加法测试

在这个过程中发现了乘除法器（多周期指令）的问题：顶层模块一直发送开始运算信号，并且输出结果没有锁定，导致屏幕上数值一直屏闪。

需要注意的是：这个问题是 ALU_DISPLAY 的问题，ALU 模块本身并没有出错。

我们通过锁定结果和进行了基于硬件开关的 start 信号逻辑使得这个问题得到解决，现在乘除法的结果如下图：



Figure 4: 乘法测试

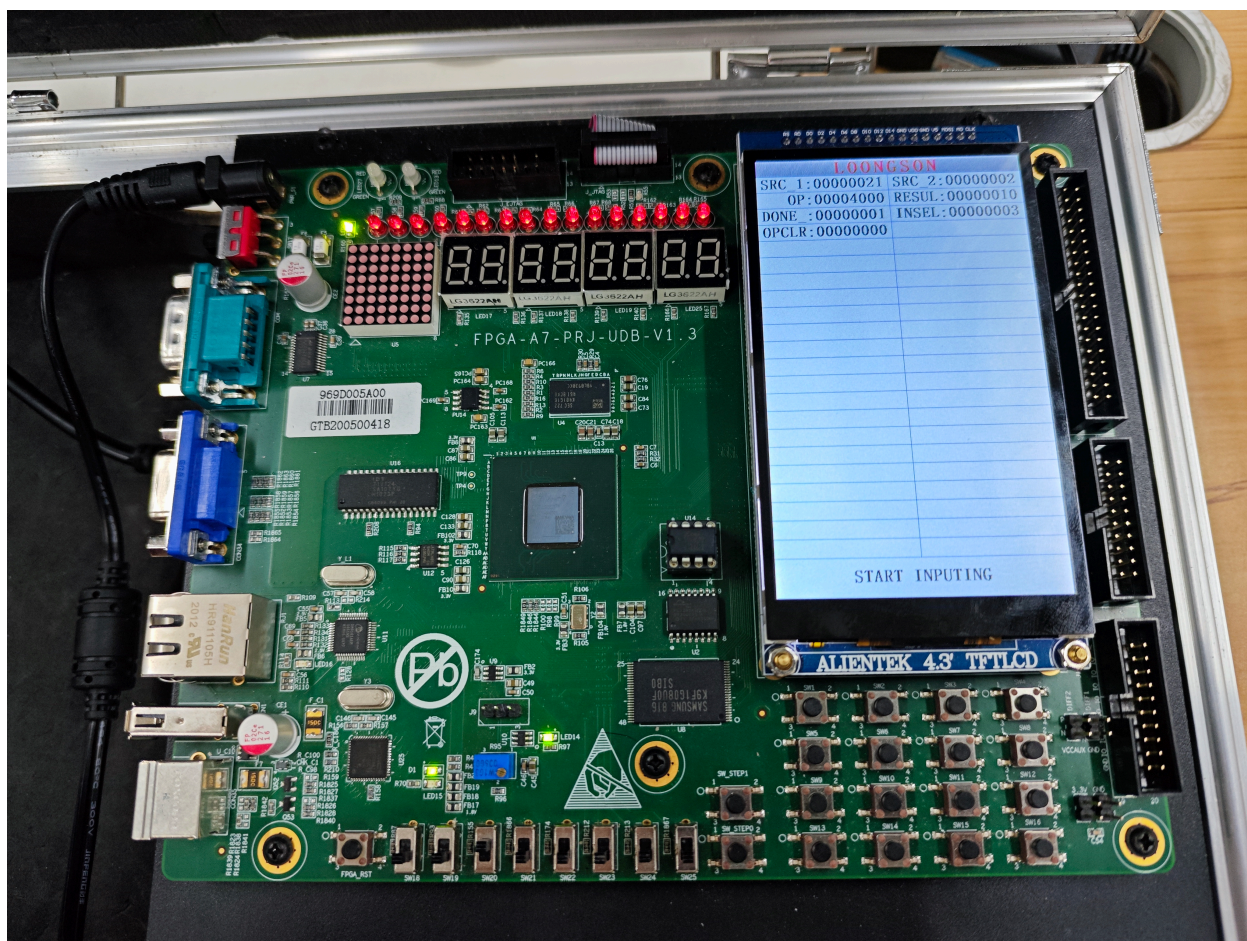


Figure 5: 除法测试

5. 结论与收获

这次实验我们完成了 32 位 ALU 的构建与验证，中途处理了相当多问题：从乘除法器的算法与构建到解决 display 模块的信号问题。

本次实验使得我们更加深入地了解 ALU 的功能与结构，以及 FPGA 的构建、使用经验，为我们后续的使用打下基础。